



Estructuras de Datos Dinámicas en C (Pt. 2)



Organización de Computadoras
Depto. Cs. e Ing. de la Comp.
Universidad Nacional del Sur



Copyright

- Copyright © **2011-2026** A. G. Stankevicius
- Se asegura la libertad para copiar, distribuir y modificar este documento de acuerdo a los términos de la **GNU** Free Documentation License, Versión 1.2 o cualquiera posterior publicada por la Free Software Foundation, sin secciones invariantes ni textos de cubierta delantera o trasera
- Una copia de esta licencia está siempre disponible en la página <http://www.gnu.org/copyleft/fdl.html>
- La versión transparente de este documento puede ser obtenida de la siguiente dirección:

<http://cs.uns.edu.ar/~ags/teaching>



Contenidos

- Concepto de puntero
- Pasaje por valor y por referencia
- Tipos de datos estructurados
- Gestión de la memoria dinámica
- Estructuras de datos dinámicas
- Concepto de posición directa e indirecta
- Fases de la compilación
- Parámetros en la línea de comandos



Memoria dinámica

- La declaración de toda variable reserva un espacio de tamaño previamente conocido en el registro de activación en curso
 - ➔ El espacio reservado para las variables de tipos de datos elementales coincide con lo retornado por la función **sizeof()** al ser aplicada a ese tipo
 - ➔ El espacio reservado para las variables de tipos de datos estructurados coincide con la suma del espacio reservado para cada uno de sus componentes
- Por otra parte, también es posible gestionar la reserva y liberación de **memoria dinámica**

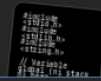


Heap vs. Stack

A ayudame a entender la diferencia entre las regiones de memoria denominadas heap y stack

Te explico las principales diferencias entre el **stack** (pila) y el **heap** (montón), que son dos regiones fundamentales de memoria en los programas:

Diferencias entre Stack y Heap - Ejemplo en C
Code



Diferencias principales entre Stack y Heap:

STACK (Pila)

- **Gestión:** Automática - el compilador se encarga
- **Velocidad:** Muy rápida - solo mover un puntero
- **Tamaño:** Limitado (normalmente 1-8 MB)
- **Orden:** LIFO (Last In, First Out) - como una pila de platos
- **Uso:** Variables locales, parámetros de funciones, direcciones de retorno
- **Limpeza:** Automática al salir del scope (ámbito)

HEAP (Montón)

- **Gestión:** Manual - el programador debe usar `malloc()` / `free()`
- **Velocidad:** Más lenta - requiere búsqueda de espacio libre
- **Tamaño:** Grande y flexible (limitado por la RAM disponible)
- **Orden:** Sin orden específico - memoria fragmentada
- **Uso:** Memoria dinámica, estructuras de datos grandes
- **Limpeza:** Manual - si no llamas `free()`, tienes memory leak

Cuándo usar cada uno:

Usa el STACK cuando:

- Las variables tienen tamaño conocido y pequeño
- Solo necesitas la memoria dentro de una función
- Quieres máxima velocidad y simplicidad

Usa el HEAP cuando:

- El tamaño se determina en tiempo de ejecución
- Necesitas estructuras de datos grandes (arrays dinámicos, listas)
- La memoria debe persistir entre diferentes funciones
- Implementas estructuras como árboles, grafos, etc.

La clave está en entender que el stack es como tu escritorio (rápido pero limitado) y el heap es como un almacén (grande pero requiere más organización).



Memoria dinámica

● **C** cuenta con las siguientes funciones de librería para la **gestionar la asignación de memoria dinámica**:

- **void* malloc(size_t)**: esta función intenta reservar la cantidad indicada de memoria dinámica
- **free(void*)**: esta función libera la porción de memoria dinámica que comienza donde se indica
- **void* realloc(void*, size_t)**: esta función reajusta al tamaño solicitado el espacio de memoria dinámica que comienza donde se indica



Memoria dinámica

- Al terminar de hacer uso de grandes estructuras dinámicas enlazadas se debe tener cuidado al liberar el espacio que ocupaban
- La invocación a **free()** sólo libera el espacio reservado por el **malloc()** que generó el puntero pasado como argumento
- ➔ Si una estructura se armó mediante múltiples invocaciones a **malloc()**, su espacio deberá ser retornado mediante múltiples invocaciones a **free()**

¡IMPORTANTE!



Memoria dinámica

```
int *i;  
char *c;  
struct persona *p;  
i = (int *) malloc(sizeof(int));  
c = (char *) malloc(sizeof(char));  
p = (struct persona *)  
    malloc(sizeof(struct persona));  
free(i);  
c = (char *) realloc(c, sizeof(char) * 9);
```



Java vs. C

- Hasta ahora los lenguajes de programación **Java** y **C** han compartido bastantes similitudes, sobre todo a nivel de sintaxis
- En este punto aparece una diferencia que estamos obligados a tener siempre en cuenta:
 - **Java** se hace cargo por completo de la recuperación de la memoria dinámica asignada a un objeto cuyo uso finalizó
 - **C**, en contraste, deja esa tarea por completo en manos del programador



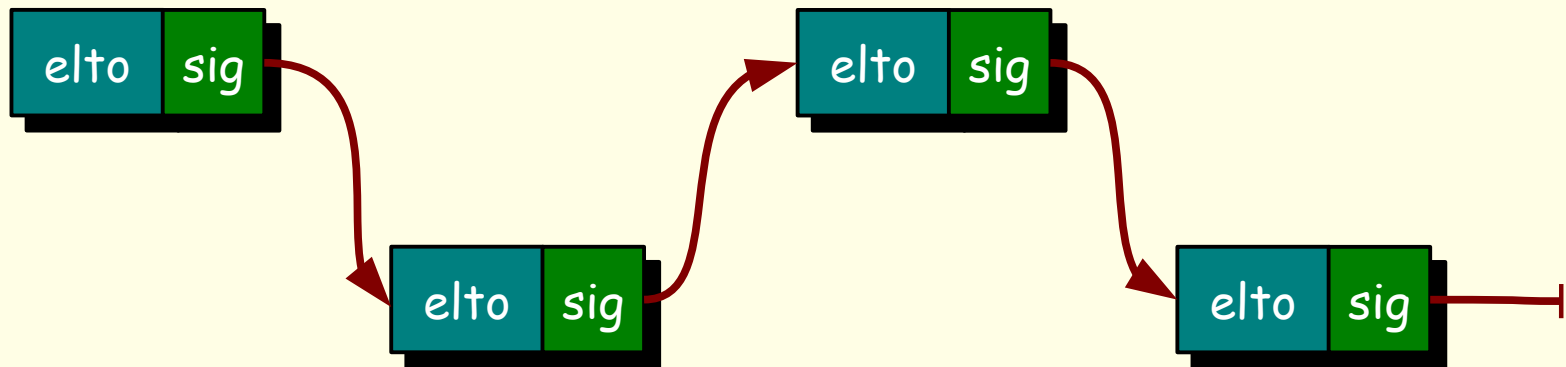
Listas

- Recordemos las principales características de la estructura de datos dinámica “lista”:
 - ➔ Es una estructura de datos dinámica
 - ➔ El espacio ocupado por sus elementos es asignado a medida que se necesite
 - ➔ Cada elemento apunta al siguiente, determinando una relación lineal
 - ➔ Puede ser simple o doblemente enlazada
 - ➔ Permite implementar pilas y colas



Listas

```
struct celda {  
    tipo elemento;  
    struct celda *sig;  
};
```



Listas

- Esta estructura de datos cierta particularidades:
 - Los elementos usualmente son todos del mismo tipo
 - Por lo general, cada celda es creada por separado invocando repetidamente a la función **malloc()**
 - Cada celda apunta a la siguiente
 - La última celda apunta a **NULL**
 - La lista completa se representa mediante un puntero al primer elemento



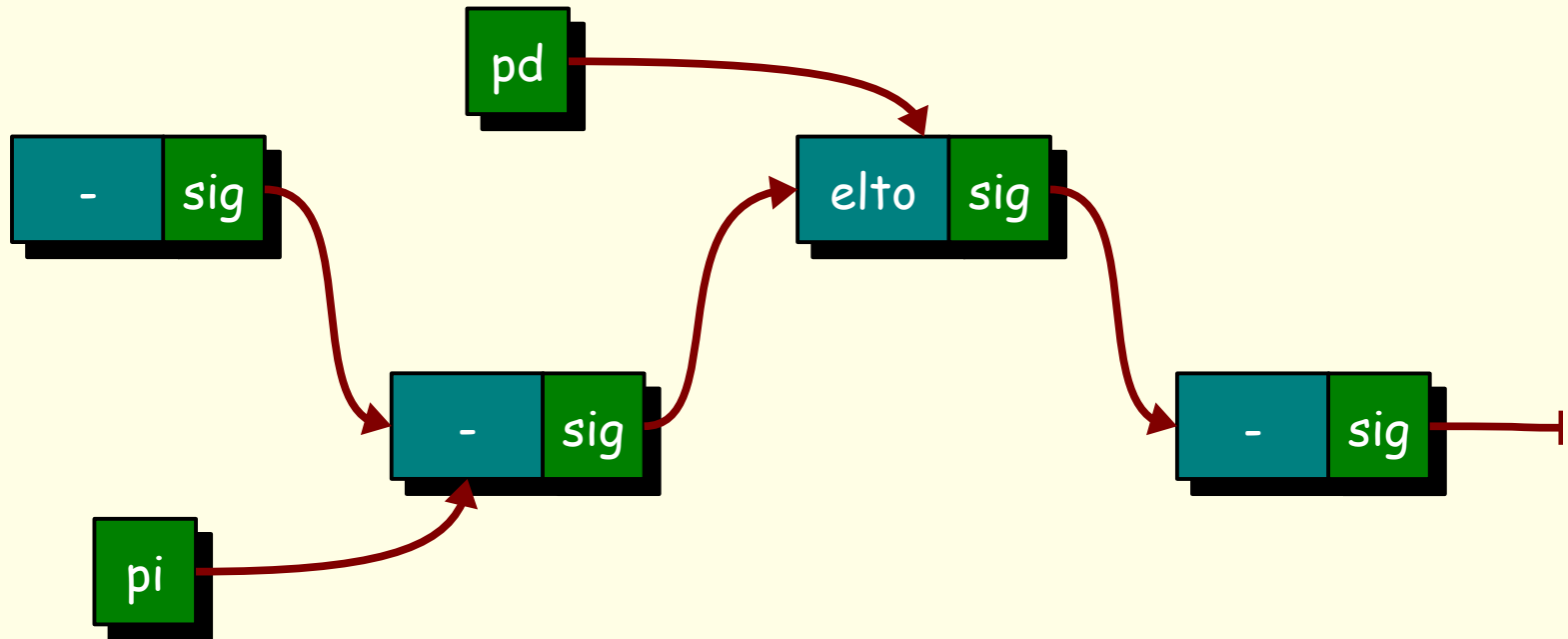
Posición directa vs. indirecta

- La **posición** de un elemento denota su ubicación dentro de la lista
- Existen principalmente dos variantes para el concepto de posición:
 - ➔ Posición directa: la posición se denota mediante un puntero a la celda conteniendo el elemento deseado
 - ➔ Posición indirecta: la posición se denota mediante un puntero a una celda conteniendo un puntero a la celda conteniendo el elemento deseado



Posición directa vs. indirecta

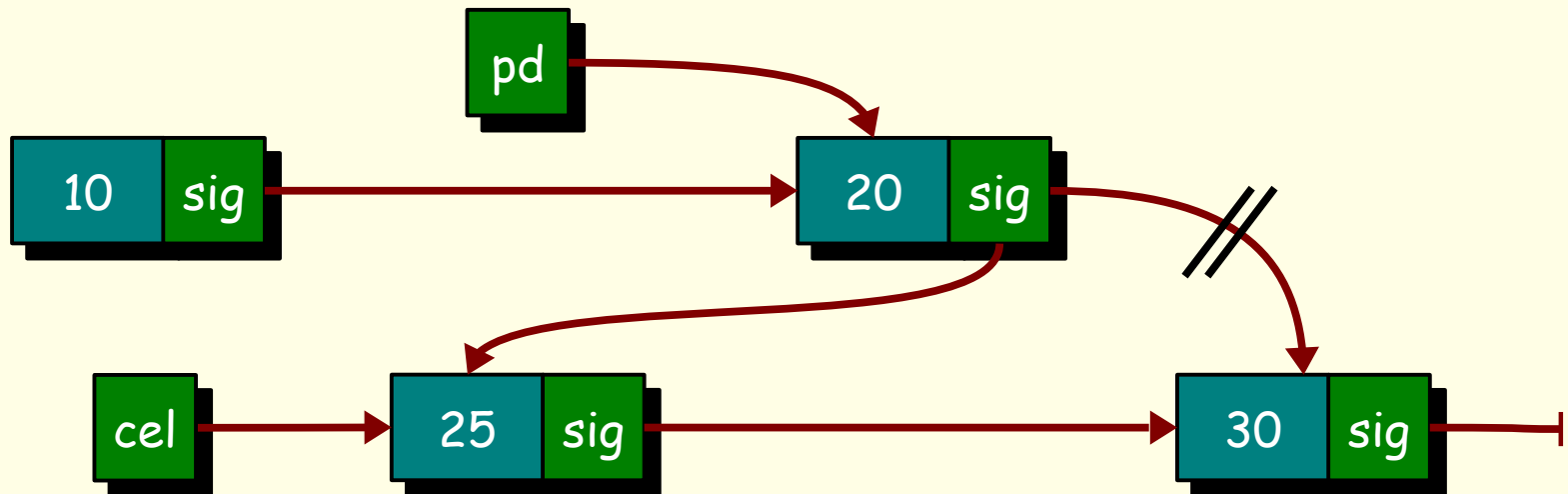
- Considerando el elemento **elto**, el puntero **pd** representa su posición directa y **pi** su posición indirecta:



Inserción en listas

- Veamos cómo agregar una nueva celda ***cel** a **continuación** del elemento en la posición ***pd** (usando el concepto de posición directa):

```
struct celda *cel, *pd;
```

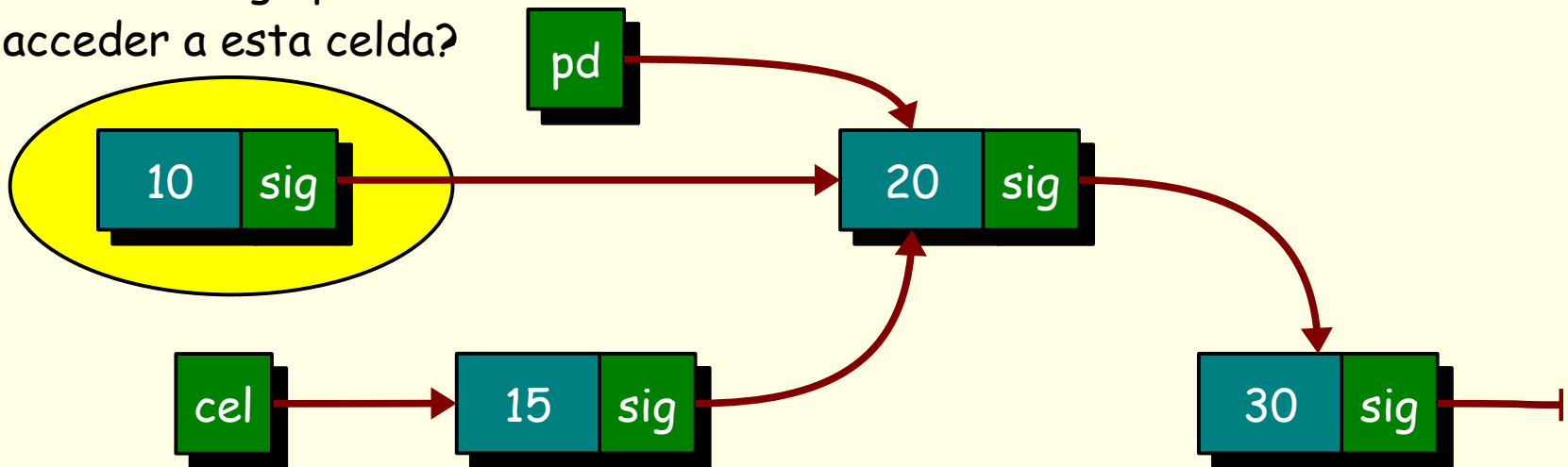


Inserción en listas

- Veamos cómo agregar una nueva celda ***cel** **antes** que el elemento en la posición ***pd** (usando el concepto de posición directa):

struct celda *cel, *pd;

¿cómo hago para acceder a esta celda?



Inserción en listas

- Para agregar un elemento a una lista antes de otro hace falta poder **acceder al elemento anterior en la lista**
- Esto se puede resolver de varias formas, siempre pagando algún costo:
 - **Recorriendo la lista** desde el principio
 - Haciendo uso de **posición indirecta** en vez de directa
 - Implementando una lista **doblemente enlazada**
- ¿Qué costo pago en cada caso?



Fases de la compilación

● Compilación (traducción):

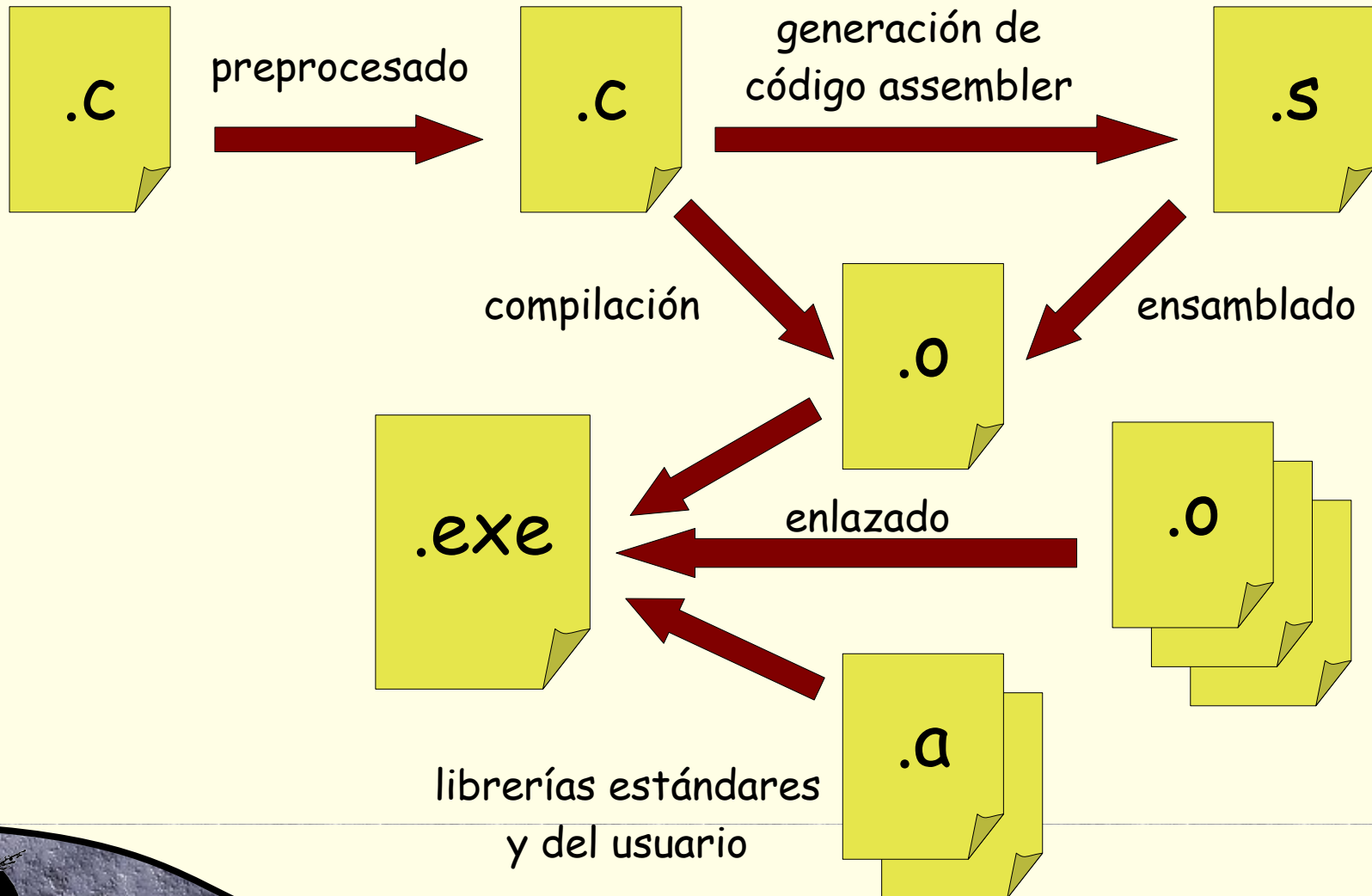
- Preprocesado
- Generación de código assembler
- Generación de código objeto

● Enlazado o vinculación:

- Enlazado con otros archivos objeto
- Enlazado con librerías estándar
- Generación del ejecutable



Fases de la compilación



Directrices al preprocesador

- Las **directrices al preprocesador** son interpretadas antes de la compilación:
 - ➔ **#define**: define una nueva constante o macro del preprocesador
 - ➔ **#include**: indica que se debe incluir el contenido de otro archivo en el punto indicado
 - ➔ **#ifdef, #ifndef**: señala el comienzo de un bloque de preprocesamiento condicionado
 - ➔ **#endif**: marca el fin de un bloque de preprocesamiento condicionado



Constantes y macros

- El preprocesador permite asociar valores constantes a ciertos identificadores que serán expandidos antes de la compilación propiamente dicha:

#define variable valor-cte

- Análogamente, las macros son funciones que han de ser expandidas antes de la compilación:

#define macro(args, ...) def-función



Constantes y macros

```
#define PI          3.14
#define CANT        5
#define AREA(rad)  PI * rad * rad
#define MAX(a, b)  (a > b ? a : b)

int main() {
    int i; float vector[CANT];
    for(i = 0; i < CANT; i++)
        vector[i] = MAX(i * 5.2, AREA(i));
}
```



Archivos de encabezamiento

- Los archivos de encabezamientos incluidos mediante la directiva **#include** suelen tener la extensión “.h”
- Las funciones declaradas en los archivos de encabezamiento no incluyen sus implementaciones
- Las variables que allí aparezcan están declaradas como **extern**, ya que su definición ha de figurar en otro archivo fuente (el “.c” asociado a ese “.h”)



Preprocesado condicional

- Para incluir código cuya compilación dependa de ciertas circunstancias, se puede hacer uso de las siguientes directivas:

#ifdef variable

<bloque de sentencias>

#endif

#ifndef variable

<bloque de sentencias>

#endif



Preprocesado condicional

```
#define DEBUG
int main() {
    int i, acc;
    for (i = 0; i < 10; i++)
        acc = i * i - 1;
    #ifdef DEBUG
        printf("fin del bucle: %d", acc);
    #endif
}
```



Conversión de tipos

- **C** cuenta con diversas funciones de librería para asistir al programador en la conversión entre tipos de datos:

- **int atoi(char*)**: traduce de strings a enteros
- **long atol(char*)**: traduce de strings a enteros largos
- **double atof(char*)**: traduce de strings a reales
- **char* itoa(char*, int)**: traduce de enteros a strings



Argumentos en línea

- En C torna simple acceder a los argumentos suministrados en la línea de comandos:

```
void main(int argc, char *argv[]) {  
    int i;  
    printf("%d argumentos", argc);  
    for (i = 1; i < argc; i++) {  
        printf("%d: %s\n", i, argv[i]);  
    }  
}
```



¿Servirá de algo?

sirve de algo en pleno siglo XXI saber procesar los argumentos recibidos por un programa en la línea de comandos?

♦ Sí, **saber procesar argumentos en la línea de comandos** es muy útil en el siglo XXI. Aunque las interfaces gráficas de usuario (GUI) son comunes, la línea de comandos sigue siendo fundamental en muchos campos.

¿Por qué es útil?

- **Automatización y scripting:** La capacidad de un programa para aceptar argumentos permite crear scripts que automatizan tareas repetitivas. Por ejemplo, puedes escribir un script para redimensionar miles de imágenes, y pasar el tamaño deseado como un argumento en lugar de tener que abrir un programa de edición para cada una.
- **Servidores y computación en la nube:** La mayoría de los servidores y entornos de nube, como AWS o Google Cloud, no tienen una interfaz gráfica. La gestión y el despliegue de aplicaciones se realizan casi exclusivamente a través de la línea de comandos. Los argumentos son esenciales para especificar qué servicio iniciar, qué configuración usar, o a qué servidor conectarse.

- **Desarrollo de software:** Herramientas de desarrollo como compiladores (GCC), sistemas de control de versiones (Git) y gestores de paquetes (npm, pip) se manejan principalmente con la línea de comandos. Los argumentos permiten a los desarrolladores especificar qué archivos compilar, qué versión de código descargar, o qué biblioteca instalar.
- **Entornos de producción:** En un entorno de producción, la estabilidad y el control son cruciales. Ejecutar programas con argumentos en la línea de comandos es más **predecible y reproducible** que usar una GUI, lo que es vital para la depuración y el mantenimiento.
- **Eficiencia:** Para usuarios avanzados y administradores de sistemas, la línea de comandos es a menudo **más rápida** que una interfaz gráfica. No se necesita hacer clic en múltiples menús y ventanas; simplemente se escribe un comando con sus argumentos y la tarea se completa instantáneamente.

En resumen, aunque pueda parecer una habilidad de la "vieja escuela", la capacidad de un programa para procesar argumentos de la línea de comandos es una base clave para la automatización, el desarrollo, y la gestión de sistemas modernos. Es una habilidad que potencia la eficiencia y el control, especialmente en entornos donde la interacción humana directa con la interfaz no es la norma.



Argumentos en línea

- Cabe acotar que la convención es que el primer argumento es el nombre del programa que se está ejecutando

```
printf("Invocado como: %s",argv[0]);
```

- Es decir, los argumentos en las las restantes posiciones (de **1** a **argc-1**), son los argumentos en la línea de comando efectivos
 - Nótese que los argumentos **se reciben como cadenas de caracteres**, incluso al tratarse de números



¿Preguntas?

